SeismoWatch Kivu

Diagrams explained: data model, alert-handling process and system architecture

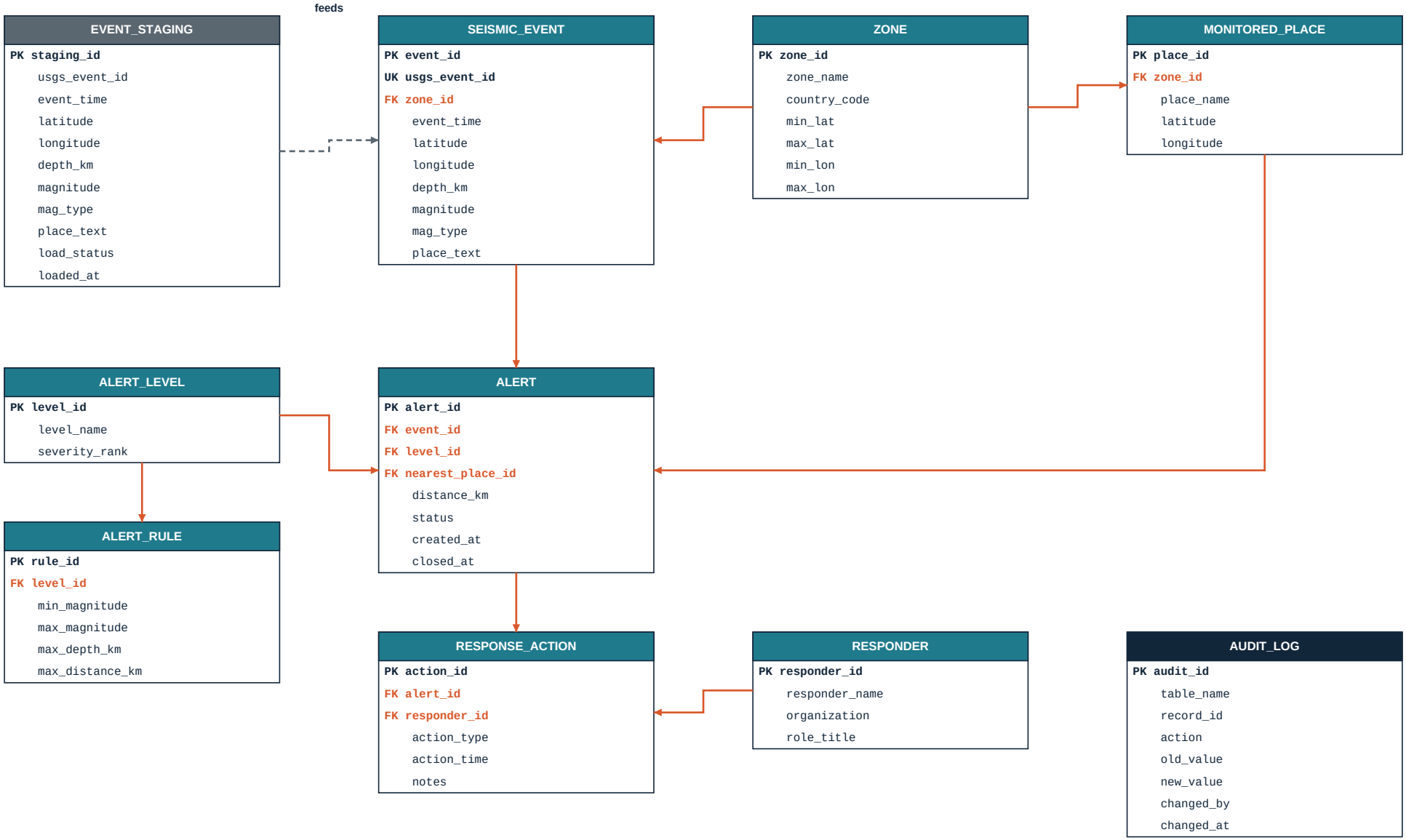
SeismoWatch Kivu is an earthquake monitoring and alerting system for the Lake Kivu region. It pulls earthquake readings from the USGS, validates them, turns the significant ones into graded alerts, and tracks each alert until a responder has acted on it and a duty officer has closed it. The three diagrams in your file show the same system from three angles:

Diagram	Question it answers
1. Entity Relationship Diagram (3NF)	What data is stored, and how are the tables related?
2. Business process (swimlane)	Who does what, and in which order, from an earthquake to a closed alert?
3. System architecture	Which technical components exist and how do they talk to each other?

How to read this document: each section shows the diagram (redrawn from the .drawio file so it fits a PDF page) followed by a plain-language walkthrough. Reading them in order works best, because the ERD names the tables, the process shows how rows in those tables change over time, and the architecture shows which program does the work.

The central idea across all three: **business rules live inside the Oracle database (PL/SQL package, triggers and scheduler jobs)**. The Python loader is deliberately dumb and only copies data in.

1. Entity Relationship Diagram (3NF)



Legend from the file: PK = primary key, FK = foreign key, UK = unique. Orange lines are one-to-many (parent to child). The dashed line shows staging feeding events. Redrawn here with straight elbow connectors.

What the ERD means

The model has **10 tables** in third normal form (3NF): every column describes only its own table's key, and repeating facts (such as a zone's name or an alert level's name) live in one place and are referenced by id. They fall into four groups.

A. Ingestion

EVENT_STAGING is a landing table for raw rows exactly as they arrive from the USGS (id, time, latitude, longitude, depth, magnitude, magnitude type, place text). It adds `load_status` and `loaded_at` so the database knows which rows are waiting, loaded or rejected. It has no foreign keys, because raw data is not trusted yet. The dashed "feeds" arrow shows that valid rows are promoted into SEISMIC_EVENT.

B. Reference and geography

Table	Purpose	Key columns
ZONE	A named rectangular area (bounding box with min/max latitude and longitude) in a country.	PK zone_id; zone_name, country_code, min_lat, max_lat, min_lon, max_lon
MONITORED_PLACE	A town, site or asset to protect (for example a city on the lake shore) located in a zone.	PK place_id; FK zone_id; place_name, latitude, longitude
ALERT_LEVEL	The severity ladder (for example Watch, Warning). severity_rank orders the levels.	PK level_id; level_name, severity_rank
ALERT_RULE	Thresholds that decide which level an event earns: magnitude range, maximum depth and maximum distance to a place.	PK rule_id; FK level_id; min_magnitude, max_magnitude, max_depth_km, max_distance_km

Keeping the thresholds in ALERT_RULE (instead of hard-coding them) means the rules can be tuned by editing data, not code.

C. Core records

Table	Purpose	Key columns
SEISMIC_EVENT	One validated earthquake. usgs_event_id is unique (UK), so the same event can never be stored twice. It points to the zone it falls in.	PK event_id; UK usgs_event_id; FK zone_id; time, location, depth, magnitude, mag_type, place_text
ALERT	A warning raised for one event at one level, with the nearest monitored place and the distance to it. It has a lifecycle: status and created_at / closed_at.	PK alert_id; FK event_id, level_id, nearest_place_id; distance_km, status, created_at, closed_at
RESPONDER	A person who handles alerts (name, organization, role title).	PK responder_id
RESPONSE_ACTION	One thing a responder did about an alert (type, time, notes). It is the link table between ALERT and RESPONDER, so one alert can have many actions and one responder can act on many alerts.	PK action_id; FK alert_id, responder_id; action_type, action_time, notes

D. Audit

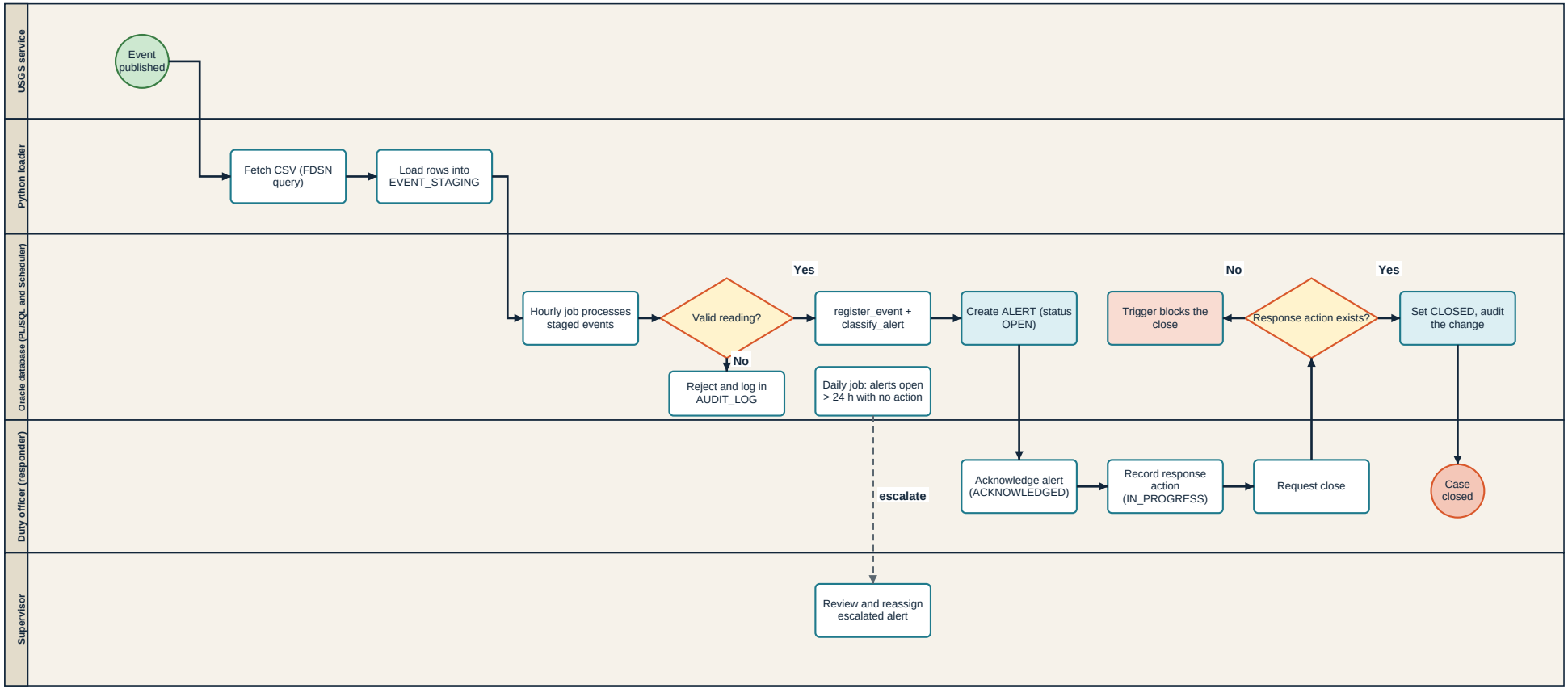
AUDIT_LOG stores who changed what and when: table name, record id, action (insert, update, delete), old value, new value, user and timestamp. The diagram note says it is filled by audit triggers on the main tables and has **no foreign keys by design**, so a log entry survives even if the row it describes is later changed or removed.

Relationships (all one-to-many, parent to child)

Parent	Child	Meaning
ZONE	SEISMIC_EVENT	A zone contains many events; each event belongs to one zone.
ZONE	MONITORED_PLACE	A zone contains many monitored places.

Parent	Child	Meaning
SEISMIC_EVENT	ALERT	An event can produce alerts.
ALERT_LEVEL	ALERT	Each alert has one level; a level is used by many alerts.
ALERT_LEVEL	ALERT_RULE	Each rule yields one level; a level can have several rules.
MONITORED_PLACE	ALERT	Each alert records its nearest monitored place.
ALERT	RESPONSE_ACTION	An alert accumulates actions as people respond.
RESPONDER	RESPONSE_ACTION	A responder performs many actions.
EVENT_STAGING	SEISMIC_EVENT	(dashed, "feeds") Valid staged rows become events; not a foreign key.

2. Alert-handling business process (swimlane)



Legend from the file: green circle = start, red circle = end, yellow diamond = decision, dashed = scheduled escalation path.

What the process means

The swimlane chart has five lanes (who or what performs each step): the **USGS service**, the **Python loader**, the **Oracle database** (PL/SQL and Scheduler), the **duty officer** and the **supervisor**. It runs from a published earthquake to a closed alert, with two guard rules built in.

Main path

#	Step	Lane	What happens
1	Event published	USGS	The USGS publishes a new earthquake (start circle).
2	Fetch CSV (FDSN query)	Python loader	The loader asks the USGS web service for recent events in CSV form.
3	Load rows into EVENT_STAGING	Python loader	Rows are copied unchanged into the staging table. No decisions are made here.
4	Hourly job processes staged events	Database	A scheduled job picks up staged rows each hour.
5	Valid reading?	Database (decision)	Each row is validated (for example plausible coordinates, depth and magnitude). **No**: the row is rejected and the rejection is logged in AUDIT_LOG. **Yes**: it continues.
6	register_event + classify_alert	Database	register_event stores the event in SEISMIC_EVENT and assigns its zone. classify_alert compares it with ALERT_RULE thresholds and the nearest monitored place.
7	Create ALERT (status OPEN)	Database	An alert is created with status OPEN.
8	Acknowledge alert (ACKNOWLEDGED)	Duty officer	The officer takes ownership; status becomes ACKNOWLEDGED.
9	Record response action (IN_PROGRESS)	Duty officer	The officer logs what was done (a RESPONSE_ACTION row); status becomes IN_PROGRESS.
10	Request close	Duty officer	The officer asks to close the alert.
11	Response action exists?	Database (decision)	**Yes**: continue to close. **No**: a trigger blocks the close, so an alert cannot be closed without evidence that someone responded.
12	Set CLOSED, audit the change	Database	Status becomes CLOSED and the change is written to AUDIT_LOG.
13	Case closed	End	End circle.

Alert status lifecycle

OPEN, then ACKNOWLEDGED, then IN_PROGRESS, then CLOSED. Each status maps to one step in the chart, so a glance at ALERT.status tells you where a case is.

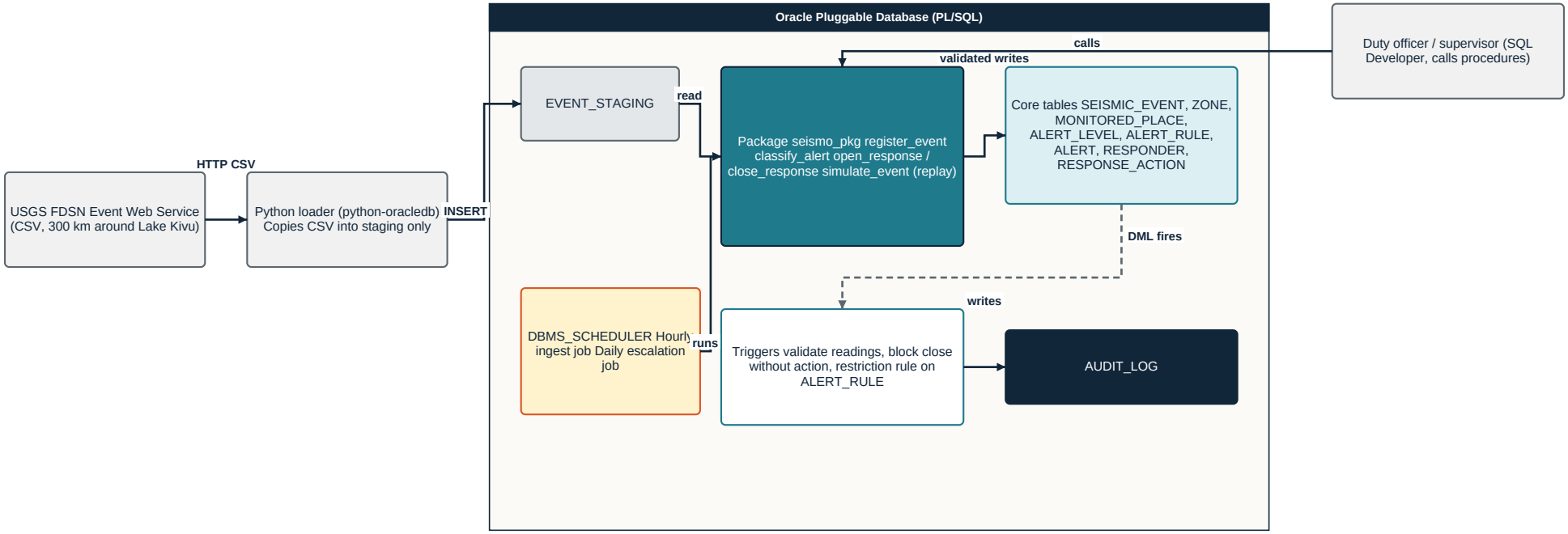
Escalation path (dashed)

A **daily job** in the database looks for alerts that have been open for more than 24 hours with no response action. It sends them down the dashed arrow to the **supervisor**, who reviews and reassigns the escalated alert. This safety net means an alert cannot sit unnoticed.

Two rules to remember

Validation gate: bad data is rejected and logged before it can create an alert. **Close guard:** no close without at least one response action. Both are enforced in the database, so they apply no matter how the data is changed.

3. System architecture



The dashed arrow from the core tables to the triggers shows that data changes (DML) fire the triggers. Redrawn here with straight elbow connectors.

What the architecture means

The diagram's own motto: **business rules stay in PL/SQL. The loader only moves data; the scheduler automates ingestion and escalation.** Everything important happens inside one Oracle pluggable database.

Component	Role
USGS FDSN Event Web Service	External data source. Returns earthquakes as CSV for a 300 km radius around Lake Kivu.
Python loader (python-oracledb)	Downloads the CSV over HTTP and inserts it into EVENT_STAGING. It copies only; it does not validate or classify.
EVENT_STAGING	Landing table inside the database (the INSERT target of the loader).
Package seismo_pkg	The business logic: register_event (validate and store an event), classify_alert (choose the level), open_response / close_response (handle responder work), and simulate_event (replay an event for testing and demos). It reads staging and makes validated writes to the core tables.
Core tables	SEISMIC_EVENT, ZONE, MONITORED_PLACE, ALERT_LEVEL, ALERT_RULE, ALERT, RESPONDER and RESPONSE_ACTION (the ERD tables).
DBMS_SCHEDULER	Runs two jobs on a timer: the hourly ingest job and the daily escalation job. Both call seismo_pkg.
Triggers	Fire on data changes: validate readings, block closing an alert without an action, and enforce a restriction rule on ALERT_RULE.
AUDIT_LOG	Receives a record from the triggers for every change.
Duty officer / supervisor	Use SQL Developer to call the package procedures. There is no separate web application in this design.

Data flow in one sentence

USGS, to the Python loader (HTTP CSV), to EVENT_STAGING (INSERT), to seismo_pkg (read), to the core tables (validated writes), which fire triggers that write AUDIT_LOG. DBMS_SCHEDULER starts seismo_pkg on a timer, and people call it by hand.

Why this design is sensible

Rules in one place. Putting validation, classification and close-guarding in the database means they cannot be bypassed by a different client. **A thin loader** is easy to replace or rerun. **Staging** keeps untrusted data apart from clean data. **The scheduler** removes the need for an external cron service. **Triggers and the audit log** give a complete history of who changed what. The trade-off is that business logic is tied to Oracle, and users need SQL tooling rather than a friendly interface.

How the three diagrams connect

Idea	ERD	Process	Architecture
Raw data lands first	EVENT_STAGING	Load rows into EVENT_STAGING	Python loader, EVENT_STAGING
Validate and classify	SEISMIC_EVENT, ALERT_RULE	Valid reading?; register_event + classify_alert	seismo_pkg
Handle the alert	ALERT, RESPONSE_ACTION, RESPONDER	Acknowledge, record action, request close	open_response / close_response
Guard rules	(constraints)	Trigger blocks the close	Triggers
History	AUDIT_LOG	Reject and log; audit the change	AUDIT_LOG
Timers	(none)	Hourly job; daily 24 h escalation	DBMS_SCHEDULER